

Chapter Overview

Chapter 07 covers build tools used in ShopVerse: Maven (lifecycle, dependency management, plugins, multi-module builds, profiles) and Gradle (task graph, Kotlin DSL, incremental builds, caching), with a side-by-side comparison and CI/CD integration patterns.

Sub-Chapter	Topic	Key Concept
07-01	Maven Fundamentals	POM, coordinates, lifecycle phases, packaging
07-02	Maven Dependency Management	Dependency scopes, BOM, exclusions, version resolution
07-03	Maven Plugins	spring-boot-maven-plugin, surefire, failsafe, jacoco, checkstyle
07-04	Maven Multi-Module Build	Parent POM, module declarations, inheritance vs aggregation
07-05	Maven Profiles	dev/prod profiles, activation, resource filtering
07-06	Gradle Fundamentals	Project, Task, DAG, build.gradle.kts, settings.gradle.kts
07-07	Gradle Dependencies & Configurations	implementation, api, testImplementation, BOM
07-08	Gradle Plugins & Tasks	spring-boot plugin, test, bootBuildImage, custom tasks
07-09	Gradle Build Cache & Incremental Builds	UP-TO-DATE, build cache, configuration cache
07-10	Maven vs Gradle Comparison	Performance, flexibility, learning curve, ecosystem
07-11	Dependency Vulnerability Scanning	OWASP dependency-check, Snyk, Trivy
07-12	CI/CD Build Integration	GitHub Actions pipeline, Docker layer caching, layer JAR

POM Structure

```
4.0.0
org.springframework.boot
spring-boot-starter-parent
3.3.0
com.shopverse
shopverse-api
1.0.0-SNAPSHOT
jar
21
1.5.5.Final
```

Maven Build Lifecycle Phases

Phase	Purpose	Key Plugin Goal Bound
validate	Validate POM structure	—
compile	Compile src/main/java	maven-compiler-plugin:compile
test	Run unit tests (src/test/java)	maven-surefire-plugin:test
package	Create JAR/WAR in target/	maven-jar-plugin:jar
verify	Run integration tests, checks	maven-failsafe-plugin:verify, jacoco:check
install	Copy to local ~/.m2 repo	maven-install-plugin:install
deploy	Push to remote repository	maven-deploy-plugin:deploy

```
# Run specific phase (all preceding phases run too)
mvn test # runs validate → compile → test
mvn package -DskipTests # build JAR without tests
mvn verify # includes integration tests
mvn spring-boot:run # start the application
mvn spring-boot:build-image # build OCI Docker image (no Dockerfile needed)
```

Dependency Scopes

Scope	Classpath	Transitive	Packaged?	ShopVerse Example
compile (default)	compile + runtime	Yes	Yes	spring-boot-starter-web, spring-data-jpa
provided	compile only	Yes	No	jakarta.servlet-api (Tomcat provides it)
runtime	runtime only	No	Yes	PostgreSQL JDBC driver (not needed at compile)
test	test only	No	No	spring-boot-starter-test, testcontainers
system	compile only	Yes	No	Local JARs (rare; avoid)
import	BOM import only	N/A	No	spring-boot-dependencies BOM

```

org.springframework.boot
spring-boot-dependencies
3.3.0
pom
import

org.springframework.boot
spring-boot-starter-data-jpa

```

Dependency Conflict Resolution

Rule	Description	Example
Nearest wins	Closer to root in dependency tree wins	If project declares Guava 32, transitive Guava 31 loses
First declaration wins	When same depth, first declared wins	Order of <dependency> in POM matters
<exclusion>	Explicitly remove transitive dep	Exclude old commons-logging pulled by legacy lib
<dependencyManagement>	Force version across all transitive	Pin Jackson to specific version throughout

Key Plugins in ShopVerse

Plugin	Version	Purpose	Key Config
spring-boot-maven-plugin	3.3.0	Package fat JAR / layered JAR / fat JAR	<layered>true</layered>
maven-compiler-plugin	3.11	Compile with Java 21; annotation processing	<release>21</release> ; MapStruct processor
maven-surefire-plugin	3.1	Run unit tests (JUnit 5)	<useModulePath>>false (JUnit 5 workaround)</useModulePath>
maven-failsafe-plugin	3.1	Run integration tests (*IT.java)	<includes>**/*IT.java</includes>
jacoco-maven-plugin	0.8.10	Code coverage; fail if <80%	<minimum>0.80</minimum> in check goal
checkstyle-maven-plugin	3.3	Code style enforcement	google_checks.xml config
versions-maven-plugin	2.16	Check for dep updates	mvn versions:display-dependency-updates
dependency-check-maven	9.0	OWASP CVE scan	<failBuildOnCVSS>7</failBuildOnCVSS>

```

org.springframework.boot
spring-boot-maven-plugin
true
shopverse/${project.artifactId}:${project.version}
paketobuildpacks/builder:base
build-info

```

ShopVerse Module Structure

shopverse/ (parent POM – aggregator)

shopverse-api/ – Spring Boot app, controllers

shopverse-domain/ – entities, repositories, domain services

shopverse-infrastructure/ – JPA impls, Redis, Kafka adapters

shopverse-common/ – shared DTOs, exceptions, utilities

shopverse-integration-tests/ – Testcontainers ITs

shopverse-common

shopverse-domain

shopverse-infrastructure

shopverse-api

shopverse-integration-tests

com.shopverse

shopverse

1.0.0-SNAPSHOT

shopverse-api

Inheritance vs Aggregation

Concept	Syntax	Effect
Inheritance	<parent> tag in child POM	Child inherits properties, plugins, dep management from parent
Aggregation	<modules> tag in parent POM	mvn install on parent builds all modules in order
Both together	Combined	Standard multi-module setup: parent manages + aggregates

```
dev
true
dev
1.0.0-dev
prod
envprod
prod

# Activate profile
mvn package -Pprod # by profile ID
mvn package -Denv=prod # by property activation
```

settings.gradle.kts

```
rootProject.name = "shopverse"
include("shopverse-api", "shopverse-domain", "shopverse-infrastructure", "shopverse-common")
```

build.gradle.kts (root)

```
plugins {
    id("org.springframework.boot") version "3.3.0" apply false
    id("io.spring.dependency-management") version "1.1.4" apply false
    kotlin("jvm") version "1.9.24" apply false
}
allprojects {
    group = "com.shopverse"
    version = "1.0.0-SNAPSHOT"
    repositories { mavenCentral() }
}
subprojects {
    apply(plugin = "java")
    java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
}
```

Gradle Task Lifecycle

Phase	Description	Example Tasks
Initialization	settings.gradle evaluated; project hierarchy created	
Configuration	All build.gradle scripts evaluated; task graph built	
Execution	Selected tasks executed in dependency order	compile, test, bootJar

```
# Common Gradle commands
./gradlew build # compile + test + jar
./gradlew test # run unit tests
./gradlew bootJar # build fat/layered JAR
./gradlew bootBuildImage # build OCI image via Cloud Native Buildpacks
./gradlew dependencies # show resolved dependency tree
./gradlew :api:test --tests "*.OrderServiceTest" # single test
```

Dependency Configurations

Configuration	Classpath	Transitive to dependencies	Most Equivalent
implementation	compile + runtime	No (not leaked)	compile (preferred)
api	compile + runtime	Yes (leaked on classpath)	compile (use sparingly)
compileOnly	compile only	No	provided
runtimeOnly	runtime only	No	runtime
testImplementation	test compile + runtime	No	test
testCompileOnly	test compile only	No	test + provided

```
dependencies {
    implementation(platform("org.springframework.boot:spring-boot-dependencies:3.3.0"))
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    runtimeOnly("org.postgresql:postgresql")
    compileOnly("org.projectlombok:lombok")
    annotationProcessor("org.projectlombok:lombok")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
    testImplementation("org.testcontainers:postgresql")
}
```

■ Use implementation over api for most dependencies. api leaks the dependency on the compile classpath of dependent modules, causing recompilation of more code on changes.


```
// Custom Gradle task
tasks.register("copyConfig") {
    from("src/main/resources/config")
    into("$buildDir/config")
}
tasks.named("processResources") { dependsOn("copyConfig") }
```

bootBuildImage with Paketo Buildpacks

```
tasks.named("bootBuildImage") {
    imageName = "shopverse/api:${project.version}"
    environment = mapOf(
        "BP_JVM_VERSION" to "21",
        "BPE_SPRING_PROFILES_ACTIVE" to "prod",
        "BPE_JAVA_OPTS" to "-XX:+UseG1GC -Xms512m -Xmx1g"
    )
    buildpacks = listOf(
        "gcr.io/paketo-buildpacks/java",
        "gcr.io/paketo-buildpacks/health-checker"
    )
}
```

Layered JAR Structure

Layer	Contents	Change Frequency	Docker Cache
dependencies	Third-party JARs	Rarely (version bumps)	Long-lived layer
spring-boot-loader	Spring Boot loader classes	Per Spring Boot version	Long-lived
snapshot-dependencies	SNAPSHOT JARs	Frequently	Shorter TTL
application	Your compiled classes + resources	Every build	No cache (always new)

```
# Layered JAR Dockerfile (efficient cache)
FROM eclipse-temurin:21-jre AS builder
WORKDIR /app
COPY target/shopverse-api-*.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=builder /app/dependencies/ ./
COPY --from=builder /app/spring-boot-loader/ ./
COPY --from=builder /app/snapshot-dependencies/ ./
COPY --from=builder /app/application/ ./
ENTRYPOINT ["java", "org.springframework.boot.loader.JarLauncher"]
```

Feature	Description	Config
UP-TO-DATE checks	Task skipped if inputs/outputs unchanged	Automatic for most built-in tasks
Local build cache	Reuse task outputs across runs on same machine	<code>org.gradle.caching=true</code> in <code>gradle.properties</code>
Remote build cache	Share cached outputs across CI machines	<code>buildCache { remote(HttpBuildCache) { url=... } }</code>
Configuration cache	Cache build configuration (task graph)	<code>org.gradle.configuration-cache=true</code>
Parallel execution	Run independent tasks concurrently	<code>org.gradle.parallel=true</code>
Worker API	Task-level parallelism within a task	WorkerExecutor API

```
# gradle.properties
org.gradle.caching=true
org.gradle.parallel=true
org.gradle.configuration-cache=true
org.gradle.jvmargs=-Xmx2g -XX:+UseG1GC
# build.gradle.kts - remote cache (CI server)
buildCache {
    local { isEnabled = true }
    remote {
        url = uri("https://gradle-cache.shopverse.internal/cache/")
        isPush = System.getenv("CI") != null // push only from CI
    }
}
```

Feature	Maven	Gradle
Build language	XML (verbose but explicit)	Groovy or Kotlin DSL (concise, programmable)
Performance	Slower (no incremental build)	Faster (incremental, build cache, parallel)
Incremental builds	No native support (timestamp check)	Yes – task input/output tracking
Build cache	No	Yes (local + remote + configuration cache)
Flexibility	Fixed lifecycle; plugin must extend	Any task graph; custom logic in DSL
Learning curve	Low (XML, well-documented)	Medium (DSL, Kotlin types)
IDE support	Excellent (IntelliJ/Eclipse)	Excellent (IntelliJ/Eclipse)
Ecosystem	Huge (Maven Central dominant)	Growing; uses Maven repositories
Multi-project builds	Supported (parent POM)	Better (composite builds, version catalogs)
Kotlin multiplatform	Limited	Native support
ShopVerse choice	Either; Maven is default in docs	Gradle preferred for large multi-module

Tool	Integration	Config	Trigger
OWASP dependency-check	Maven/Gradle plugin	failBuildOnCVSS=7	mvn verify / gradle check
Snyk	CLI + GitHub Action	snyk test --severity-threshold=high	PR check + scheduled daily
Trivy	Docker/GitHub Action	--severity HIGH,CRITICAL	Docker image scan in CI
GitHub Dependabot	GitHub native	dependabot.yml	PR auto-creation for updates

```

org.owasp
dependency-check-maven
9.0.9
7
dependency-check-suppressions.xml
check
# GitHub Actions - Trivy image scan
- name: Scan Docker image
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: "shopverse/api:${{ github.sha }}"
    format: "sarif"
    severity: "HIGH,CRITICAL"
    exit-code: "1"

```

GitHub Actions Maven Pipeline

```
name: ShopVerse CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: "21", distribution: "temurin" }
      - name: Cache Maven repository
        uses: actions/cache@v4
        with:
          path: ~/.m2/repository
          key: ${ runner.os }-maven-${ hashFiles("**/pom.xml") }
      - name: Build and test
        run: mvn -B verify
      - name: JaCoCo coverage report
        uses: codecov/codecov-action@v4
      - name: Build Docker image
        run: mvn spring-boot:build-image -DskipTests
      - name: Push to registry
        run: docker push shopverse/api:${ github.sha }
```

■ Do

- ✓ Use BOM imports (spring-boot-dependencies) to manage transitive dependency versions.
- ✓ Pin plugin versions explicitly in pom.xml/build.gradle – reproducible builds.
- ✓ Use layered JAR Dockerfile for optimal Docker layer caching (dependencies layer rarely changes).
- ✓ Run OWASP dependency-check in CI pipeline; fail build on CVSS ≥ 7 .
- ✓ Enable Gradle build cache (local + remote in CI) for faster incremental builds.
- ✓ Use gradle.properties for JVM tuning: org.gradle.jvmargs=-Xmx2g.
- ✓ Separate unit tests (surefire) from integration tests (failsafe) for faster feedback loops.
- ✓ Use mvn -B (batch mode) in CI to suppress download progress spam.
- ✓ Use Gradle version catalogs (libs.versions.toml) for centralised dependency version management.

■ Anti-Patterns

- Omitting plugin versions – builds differ across environments when plugins update.
- Using compile scope in Gradle (deprecated) – use implementation.
- api instead of implementation for most Gradle deps – leaks implementation details, causes over-compilation.
- Not caching ~/.m2 or ~/.gradle in CI – downloading all dependencies on every build run.
- Fat JAR without layer tools – every code change rebuilds all dependency layers in Docker.
- Ignoring dependency CVE scan output – security vulnerabilities ship undetected.
- Hardcoded version numbers scattered across modules – use BOM or version catalog.
- maven-compiler-plugin without explicit Java version – inherits from environment, not POM.

Sub-Chapter	Key Point	One-Liner
07-01 Lifecycle	mvn verify	Runs validate→compile→test→package→verify (includes ITs)
07-02 BOM	<scope>import</scope>	Import version recommendations without adding the dependency
07-03 Plugins	spring-boot:build-image	OCI image via Paketo buildpacks; no Dockerfile needed
07-04 Multi-module	<modules> in parent	Build order derived from inter-module dependencies
07-05 Profiles	mvn -Pprod	Activate profile; changes properties, plugins, deps
07-06 Gradle	./gradlew build	compile + test + jar in one command
07-07 implementation	Annotation	implementation avoids leaking dep to consumers
07-08 Layered JAR	4 layers	dependencies / loader / snapshot-deps / application
07-09 Build cache	org.gradle.caching=true	Skip re-running tasks with unchanged inputs
07-10 Maven vs Gradle	Gradle faster	Build cache + incremental = 5–20× faster on large repos
07-11 OWASP	FailBuildOnCVSS=7	Fail CI if any dependency has HIGH/CRITICAL CVE
07-12 CI	Cache ~/.m2	Avoid re-downloading all deps on every CI run

Interview Quick-Check

Question	Concise Answer
Maven lifecycle vs phase vs goal?	Lifecycle: set of phases (default, clean, site). Phase: step in lifecycle. Goal: plugin action; bound to lifecycle.
BOM vs parent POM?	Both manage versions. Parent POM also inherits plugin config. BOM (import scope) only manages dependencies.
Gradle implementation vs api?	implementation: dep not on compile classpath of dependents (faster recompile). api: dep is leaked; on compile classpath.
Layered JAR benefit?	Dependencies layer rarely changes → Docker caches it. Only application layer rebuilt on code changes.
Why build cache?	Tasks with identical inputs produce identical outputs. Gradle stores outputs by input hash; replays if hash matches.